

作业 HW-3 实验报告

姓名：毛星博 学号：2351520 日期：2024.11.7

1. 涉及数据结构和相关背景

1.1 二叉树

二叉树是一种树形数据结构，每个节点最多有两个子节点，通常称为左子节点和右子节点。二叉树广泛应用于计算机科学，包括数据存储、搜索以及表达式解析等。

二叉树的基本术语：

根节点 (Root): 二叉树的最顶层节点，不存在父节点。

子节点 (Child): 直接连接在某个节点下方的节点，分为左子节点和右子节点。

父节点 (Parent): 向下连接到子节点的节点。

叶子节点 (Leaf): 没有子节点的节点，即二叉树的末端节点。

深度 (Depth): 从根节点到某一节点的路径长度，根节点的深度为 0。

高度 (Height): 从某一节点到最远叶子节点的路径长度，叶子节点的高度为 0。

二叉树的基本操作：

初始化: 创建一个空的二叉树。

插入 (Insert): 将新节点插入到二叉树中，以保持二叉树的特性。

删除 (Delete): 删除指定节点并重新调整树结构，保持二叉树特性。

查找 (Search): 在二叉树中查找某个指定值的节点。

遍历 (Traversal): 以特定的顺序访问二叉树中的每个节点，主要有三种方式：

前序遍历 (Pre-order Traversal): 访问根节点，遍历左子树，遍历右子树。

中序遍历 (In-order Traversal): 遍历左子树，访问根节点，遍历右子树。

后序遍历 (Post-order Traversal): 遍历左子树，遍历右子树，访问根节点。

1.1.1 二叉搜索树 (Binary Search Tree)

二叉搜索树是一种特定类型的二叉树，其中每个节点的值都大于它的左子节点的值，且小于它的右子节点的值。这一特性使得二叉搜索树非常适合用于快速查找。

二叉搜索树的基本操作：

插入: 根据节点值大小确定插入位置，确保树的特性得以保持。

查找: 通过比较值，定位到相应的节点。

删除: 删除节点时需要考虑三种情况：

被删除节点是叶子节点。

被删除节点有一个子节点。

被删除节点有两个子节点，此时需找到一个替代节点（通常为右子树的最小节点或左子树的最大节点）。

1.1.2 完全二叉树 (Complete Binary Tree)

完全二叉树是所有层都填满，且最后一层从左到右填充的二叉树。这意味着所有的叶子节点都在同一层，或在倒数第二层。

完全二叉树的特征：

除了最后一层外，其他层都必须完全充满。

可以通过数组进行存储，适合用于堆的实现。

1.1.3 平衡二叉树 (Balanced Binary Tree)

平衡二叉树是一种特殊的二叉搜索树，确保其左右子树的高度差不超过 1。这种结构可以优化查找、插入和删除操作的效率。

常见的平衡二叉树：

AVL 树: 在每个节点上，维护一个平衡因子（左子树高度 - 右子树高度），以确保平衡性。

红黑树: 通过颜色标记以及一系列的旋转操作，保证树的平衡。

二叉树的应用：

数据存储和检索

表达式解析和求解（如语法树）

实现优先队列（如堆）

图的表示（如邻接表）

1.2 有序树

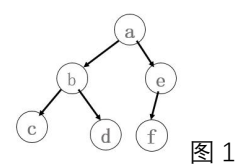
有序树的每个节点可以有多个子节点，其余操作和二叉树类似。

2. 实验内容

2.1 二叉树的非递归遍历

2.1.1 问题描述

二叉树的非递归遍历可通过栈来实现。例如对于由 abc##d##ef###先序建立的二叉树，如下图 1 所示，中序非递归遍历（参照课本 p131 算法 6.3）可以通过如下一系列栈的入栈出栈操作来完成：push(a) push(b) push(c) pop pop push(d)pop pop push(e) push(f) pop pop。



如果已知中序遍历的栈的操作序列，就可唯一地确定一棵二叉树。请编程输出该二叉树的后序遍历序列。

提示：本题有多种解法，仔细分析二叉树非递归遍历过程中栈的操作规律与遍历序列的关系，可将二叉树构造出来。

2.1.2 基本要求

输入：第一行一个整数 n，表示二叉树的结点个数。

接下来 2n 行，每行描述一个栈操作，格式为：push X 表示将结点 X 压入栈中，pop 表示从栈中弹出一个结点。(X 用一个字符表示)

输出：一行，后序遍历序列。

2.1.3 数据结构设计

```
// 定义树节点
struct TreeNode {
```

```

    char value;

    TreeNode* lchild;

    TreeNode* rchild;

    TreeNode(char val) : value(val), lchild(nullptr), rchild(nullptr) {} // 构造函数
};

// 定义栈节点
struct StackNode {

    TreeNode* treeNode;

    StackNode* next;

    StackNode(TreeNode* node) : treeNode(node), next(nullptr) {}
};

// 自定义栈
class Stack {

public:

    StackNode* topNode; // 栈顶节点指针

    Stack() : topNode(nullptr) {} // 构造函数

    // 入栈操作

    void push(TreeNode* node) {

        StackNode* newNode = new StackNode(node);

        newNode->next = topNode;

        topNode = newNode;

    }

    // 出栈操作

    TreeNode* pop() {

        if (topNode == nullptr) {

            return nullptr;

        }

        StackNode* nodeToPop = topNode;

        topNode = topNode->next;

        TreeNode* poppedNode = nodeToPop->treeNode;

        delete nodeToPop; // 释放内存

        return poppedNode;

    }

    // 判断栈是否为空

    bool isEmpty() { return topNode == nullptr; }

};

```

2.1.4 功能说明（函数、类）

```

TreeNode* buildTree(int count) // count = 2n
{

    struct VisitNode {

        TreeNode* node;

```

```

    bool visited;

    VisitNode(TreeNode* node) : node(node), visited(false) {}
};

stack<VisitNode*> nodeStack; // 节点栈
string operation;
char value;
TreeNode* root = nullptr;
VisitNode* current = nullptr;
for (int i = 0; i < count; i++) {
    cin >> operation;
    if (operation == "push") {
        // 提取节点值
        cin >> value;
        TreeNode* newNode = new TreeNode(value);
        if (root == nullptr) {
            root = newNode; // 根节点为空，则将当前节点设置为根节点
            current = new VisitNode(root); // 记录当前节点
        }
        else {
            // 如果栈不为空，设置为当前节点的左或右子节点
            if (current->visited == false)
                current->node->lchild = newNode;
            else
                current->node->rchild = newNode;
        }
        nodeStack.push(new VisitNode(newNode)); // 将当前节点推入栈中
        current = nodeStack.top(); // 将当前节点设置为新节点
    }
    else if (operation == "pop") {
        current = nodeStack.top(); // 弹出栈顶节点
        nodeStack.pop(); // 弹出栈顶节点
        current->visited = true; // 标记当前节点已访问
    }
}

return root; // 返回构建好的树的根节点
}

// 后序遍历
void postOrderTraversal(TreeNode* node) {
    if (node == nullptr)
        return;

    postOrderTraversal(node->lchild);
    postOrderTraversal(node->rchild);
}

```

```
cout << node->value ; // 先访问左子树，再访问右子树，最后访问节点本身
}
```

2.1.5 调试分析（遇到的问题 and 解决方法）

在实现非递归遍历时，遇到的问题主要集中在栈的操作顺序和节点的连接上。通过仔细分析栈的入栈和出栈顺序，确保每个节点的左右子树能够正确连接。调试过程中，使用了打印语句来跟踪栈的状态和当前节点的值，帮助确认树的结构是否正确。最终，通过多次测试不同的栈操作序列，确保了程序的正确性。

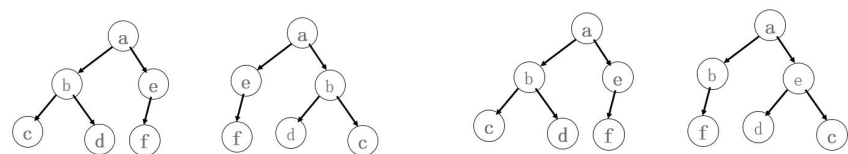
2.1.6 总结和体会

通过这次实验，我深入理解了二叉树的非递归遍历方法及其实现细节。栈的使用使得遍历过程更加灵活，避免了递归带来的栈溢出问题。此外，构建树的过程让我认识到树结构的动态性和复杂性，增强了我对数据结构的理解。

2.2 二叉树的同构

2.2.1 问题描述

给定两棵树 T1 和 T2。如果 T1 可以通过若干次左右孩子互换变成 T2，则我们称两棵树是“同构”的。例如图 1 给出的两棵树就是同构的，因为我们把其中一棵树的结点 a、b、e 的左右孩子互换后，就得到另外一棵树。而图 2 就不是同构的。



现给定两棵树，请你判断它们是否是同构的。并计算每棵树的深度。

2.2.2 基本要求

输入:第一行是一个非负整数 N1，表示第 1 棵树的结点数；
随后 N 行，依次对应二叉树的 N 个结点（假设结点从 0 到 N-1 编号），每行有三项，分别是 1 个英文大写字母、其左孩子结点的编号、右孩子结点的编号。如果孩子结点为空，则在相应位置上给出“-”。给出的数据间用一个空格分隔。
接着一行是一个非负整数 N2，表示第 2 棵树的结点数；
随后 N 行同上描述一样，依次对应二叉树的 N 个结点。
输出:共三行。
第一行，如果两棵树是同构的，输出“Yes”，否则输出“No”。
后面两行分别是两棵树的深度。

2.2.3 数据结构设计

```
const int MAXN = 20000; // 定义最大节点数

vector<int> tree1[MAXN]; // 用于存储树 1 的邻接表
vector<int> tree2[MAXN]; // 用于存储树 2 的邻接表
```

```

char value1[MAXN];      // 存储树 1 节点的值
char value2[MAXN];      // 存储树 2 节点的值
bool isChild1[MAXN];    // 标记树 1 的节点是否为子节点
bool isChild2[MAXN];    // 标记树 2 的节点是否为子节点

```

2.2.4 功能说明（函数、类）

```

// 计算树的深度
int GetDepth(int root, int index) {
    int maxDepth = 0
    如果 index 等于 1:
        如果 tree1[root] 为空:
            返回 0
        对于 tree1[root] 中的每个子节点 child:
            //计算 child 的深度
            childDepth = GetDepth(child, index)
            如果 childDepth 大于 maxDepth:
                更新 maxDepth 为 childDepth
    否则如果 index 等于 2:
        如果 tree2[root] 为空:
            返回 0
        对于 tree2[root] 中的每个子节点 child:
            //计算 child 的深度
            childDepth = GetDepth(child, index)
            如果 childDepth 大于 maxDepth:
                更新 maxDepth 为 childDepth
    return maxDepth + 1
}

// 判断两棵树是否同构
bool Isomorphic(int root1, int root2) {
    //将 tree1[root1] 的子节点存储在列表 child1 中并排序
    //将 tree2[root2] 的子节点存储在列表 child2 中并排序
    vector<int> child1(tree1[root1].begin(), tree1[root1].end());
    vector<int> child2(tree2[root2].begin(), tree2[root2].end());
    sort(child1.begin(), child1.end());
    sort(child2.begin(), child2.end());

    如果 value1[root1] 不等于 value2[root2]:
        return false

    如果 child1 的大小不等于 child2 的大小:
        return false

    对于 child1 和 child2 中的每个对应节点 i:
        return value1[child1[i]] 不等于 value2[child2[i]]:

```

```

        返回 false

    对于 child1 和 child2 中的每个对应节点 i: // 递归判断子树是否同构
        return Isomorphic(child1[i], child2[i]) 返回 false:

        返回 false

    return true
}

```

2.2.5 调试分析（遇到的问题和解决方法）

在判断两棵树是否同构的过程中，主要问题在于如何处理子树的比较。通过对比子节点的顺序和数量，确保了同构的判断逻辑。调试时，使用了多组测试数据，包括边界情况（如空树和只有一个节点的树）

2.2.6 总结和体会

这道题让我体会到了树结构的灵活性和复杂性。同构的判断不仅需要考虑节点的值，还需要考虑结构的相似性。

2.3 感染二叉树需要的总时间

2.3.1 问题描述

给你一棵二叉树的根节点 root，二叉树中节点的值互不相同。另给你一个整数 start。在第 0 分钟，感染将会从值为 start 的节点开始爆发。

每分钟，如果节点满足以下全部条件，就会被感染：

- 1.节点此前还没有感染。
- 2.节点与一个已感染节点相邻。

返回感染整棵树需要的分钟数。

2.3.2 基本要求

输入：第一行包含两个整数 n 和 start

接下来包含 n 行，描述 n 个节点的左、右孩子编号

0 号节点为根节点， $0 \leq \text{start} < n$

输出：一个整数，表示感染整棵二叉树所需要的时间

2.3.3 数据结构设计

```

// 定义二叉树结点结构
typedef struct BiTreeNode {
    TElemType value; // 树结点的值
    bool visited; // 标记该结点是否被访问过
    BiTreeNode* lchild; // 指向左子结点
    BiTreeNode* rchild; // 指向右子结点
    BiTreeNode* parent; // 指向父结点

    // 构造函数
}

```

```

        BiTreeNode(TElemType val)
            : value(val), lchild(nullptr), rchild(nullptr) , parent(nullptr) , visited(false) {}
    }* BiTree;

vector<BiTree> nodes = CreateTree(n); // 创建二叉树

```

2.3.4 功能说明（函数、类）

```

// 创建二叉树的函数
vector<BiTree> CreateTree(int n) {
    vector<BiTree> nodes(n); // 用于存储节点指针

    for (int i = 0; i < n; ++i) {
        nodes[i] = new BiTreeNode(i); // 创建节点并初始化编号
    }

    for (int i = 0; i < n; ++i) {
        int left, right;
        cin >> left >> right; // 输入左、右孩子编号

        if (left != -1) {
            nodes[i]->lchild = nodes[left]; // 将左孩子指向该节点
            nodes[left]->parent = nodes[i]; // 将该节点的父节点指向左孩子
        }

        if (right != -1) {
            nodes[i]->rchild = nodes[right]; // 将右孩子指向该节点
            nodes[right]->parent = nodes[i]; // 将该节点的父节点指向右孩子
        }
    }

    return nodes; // 返回根节点
}

// 实现广度优先搜索算法，用于在二叉树中查找节点，并计算感染时间
int BFS(vector<BiTree> nodes, int start) {
    queue<BiTree> q;
    q.push(nodes[start]);
    int time = -1;

    while (!q.empty()) {
        int currentLevelSize = q.size(); // 当前层的节点数

        for (int i = 0; i < currentLevelSize; i++) {
            BiTree node = q.front();
            q.pop();

            if (node->visited == false) {
                node->visited = true; // 标记 node 为已访问

                if (node->lchild != nullptr && node->lchild->visited == false) {
                    q.push(

```

```

        node->lchild); // 如果 node 的左子节点存在且未被访问,将 node 的左子节点入队 q
    }
    if (node->rchild != nullptr&& node->rchild->visited == false) {
        q.push(node->rchild); // 如果 node 的右子节点存在且未被访问,将 node 的右子节点入队 q
    }
    if (node->parent != nullptr&& node->parent->visited == false) {
        q.push(node->parent); // 如果 node 的父节点存在且未被访问,将 node 的父节点入队 q
    }
}
}

time++; // 每处理完一层, time 自增
}

return time;
}

```

2.3.5 调试分析（遇到的问题和解决方法）

在实现广度优先搜索（BFS）算法时，遇到的问题主要是如何正确标记已访问节点并处理相邻节点的感染。通过调试，我发现需要确保每个节点只被访问一次，以避免无限循环。使用队列的方式有效地管理了当前层的节点，确保了时间计算的准确性。

2.3.6 总结和体会

这道题让我认识到广度优先搜索在树结构中的应用，特别是在处理层级关系时的有效性。通过模拟感染过程，我对树的遍历和节点之间的关系有了更深入的理解。

2.4 树的重构

2.4.1 问题描述

树木在计算机科学中有许多应用。也许最常用的树是二叉树，但也有其他的同样有用的树类型。其中一个例子是有序树（任意节点的子树是有序的）。

每个节点的子节点数是可变的，并且数量没有限制。一般而言，有序树由有限节点集合 T 组成，并且满足：

1. 其中一个节点置为根节点，定义为 $\text{root}(T)$;
2. 其他节点被划分为若干子集 $T_1, T_2, \dots, T_m$, 每个子集都是一个树。

同样定义 $\text{root}(T_1), \text{root}(T_2), \dots, \text{root}(T_m)$ 为 $\text{root}(T)$ 的孩子，其中 $\text{root}(T_i)$ 是第 i 个孩子。节点 $\text{root}(T_1), \dots, \text{root}(T_m)$ 是兄弟节点。

通常将一个有序树表示为二叉树是更加有用的，这样每个节点可以存储在相同内存空间中。有序树到二叉树的转化步骤为：

1. 去除每个节点与其子节点的边
2. 对于每一个节点，在它与第一个孩子节点之间添加一条边，作为该节点的左孩子
3. 对于每一个节点，在它与下一个兄弟节点之间添加一条边，作为该节点的右孩子

2.4.2 基本要求

输入：由多行组成，每一行都是一棵树的深度优先遍历时的方向。其中 d 表示下行(down)，u 表示上行(up)。

可以假设每棵树至少含有 2 个节点，最多 10000 个节点。

输出：对每棵树，打印转化前后的树的深度，采用以下格式 Tree t: h1 => h2。其中 t 表示样例编号(从 1 开始)，h1 是转化前的树的深度，h2 是转化后的树的深度。

2.4.3 数据结构设计

```
typedef int Status;
typedef int TElemType;

// 二叉树节点结构体
typedef struct BiTNode {
    TElemType data;          // 节点存储的数据
    BiTNode *lchild, *rchild; // 左右子树指针
}* BiTree;

// 顺序栈模板结构体
template <class SElemType>
struct SqStack {
private:
    SElemType* base;        // 栈底指针
    SElemType* top;         // 栈顶指针
    int stacksize;          // 栈的大小
public:
    SqStack();              // 构造空栈
    ~SqStack();             // 销毁已有的栈
    Status Pop(SElemType& e); // 弹出栈顶元素
    Status Push(SElemType e); // 新元素入栈
};
```

2.4.4 功能说明（函数、类）

```
/**
 * 创建一棵二叉树并计算最大深度。
 * @param str 输入的描述字符串，其中'd'表示向下移动，'u'表示向上移动。
 * @param maxdepth 用于记录树的最大深度。
 * @return 返回树的根节点。
 */
BiTree createBiTree(string str, int& maxdepth) {
    // 初始化当前深度和最大深度
    int depth1 = 0;
    maxdepth = 0;
```

```

// 定义一个访问节点结构，用于标记节点是否已访问
struct VisitNode {
    BiTree node;

    bool flag;

    // 构造函数初始化节点和访问标记
    VisitNode(BiTree node) : node(node), flag(false) {}
};

// 使用栈来辅助构建二叉树
stack<VisitNode*> stack;

BiTree root = NULL;

// 创建根节点
BiTree p = new (nothrow) BiTreeNode;
if (!p) exit(-1); // 如果分配失败，则退出
p->lchild = p->rchild = NULL;
root = p; // 将根节点赋值给 root

// 创建当前访问节点，并将其压入栈中
VisitNode* cur = new (nothrow) VisitNode(p);
stack.push(cur);

// 遍历输入字符串
for (unsigned int i = 0; i < str.length(); i++) {
    if (str[i] == 'd') { // 若字符为 'd'，表示向下进入子节点
        depth1++; // 增加当前深度
        maxdepth = max(maxdepth, depth1); // 更新最大深度
        // 创建新的树节点
        p = new (nothrow) BiTreeNode;
        if (!p) exit(-1); // 如果分配失败，则退出
        p->lchild = p->rchild = NULL;
        // 根据标记决定将新节点作为左子节点或右子节点
        if (cur->flag == false)
            cur->node->lchild = p;
        else
            cur->node->rchild = p;
        // 创建新的访问节点，并压入栈中
        cur = new (nothrow) VisitNode(p);
        stack.push(cur);
    } else if (str[i] == 'u') { // 若字符为 'u'，表示向上返回父节点
        depth1--; // 减少当前深度
        cur = stack.top(); // 获取栈顶元素
        stack.pop(); // 弹出栈顶元素
        cur->flag = true; // 标记为已访问
    }
}
}

```

```
// 返回构建的二叉树的根节点  
  
return root;  
  
}
```

2.4.5 调试分析（遇到的问题和解决方法）

在实现树的重构过程中，主要挑战在于如何正确处理节点的上下关系。通过使用栈来管理当前节点的状态，确保了树的结构能够正确重建。

2.4.6 总结和体会

通过这道题，我对树的重构过程有了更深入的理解。树的结构变化和节点关系的处理让我认识到数据结构的灵活性和复杂性。

2.5 最近公共祖先

2.5.1 问题描述

给出一颗多叉树，请你求出两个节点的最近公共祖先。

一个节点的祖先节点可以是该节点本身，树中任意两个节点都至少有一个共同祖先，即根节点。

2.5.2 基本要求

输入：输入数据包含 T 个测试样本，每个样本 i 包含 Ni 个节点和 Ni-1 条边和 Mi 个问题，树中节点从 1 到 Ni 编号

输入第一行是测试样本数 T

每个测试样本 i 第一行为两个整数 Ni 和 Mi

接下来 Ni-1 行，每行 2 个整数 a、b，表示 a 是 b 的父节点

接下来 Mi 行，每行两个整数 x、y，表示询问 x 和 y 的共同祖先输出

对于每一个询问输出一个整数，表示共同祖先的编号

输出：对于每一个询问输出一个整数，表示共同祖先的编号

2.5.3 数据结构设计

```
const int MAXN = 1000; // 假设最大节点数  
const int MAX_LOG = 10; // log2(1000)约为 10  
  
vector<int> tree[MAXN + 1]; // 用于存储树的邻接表表示  
int parent[MAXN + 1][MAX_LOG]; // parent[i][j]表示 i 节点的第 2^j 个祖先  
int depth[MAXN + 1]; // 存储每个节点的深度  
bool isChild[MAXN + 1]; // isChild[i]表示 i 是否为某个节点的孩子，用于判断是否为根节点
```

2.5.4 功能说明（函数、类）

```
// 深度优先搜索函数，用于填充父节点信息和计算节点深度  
void dfs(int node, int par) {
```

```

parent[node][0] = par; // 父节点
for (int i = 1; i < MAX_LOG; ++i) {
    if (parent[node][i - 1] != -1) // 如果父节点存在
        parent[node][i] = parent[parent[node][i - 1]][i - 1]; // 则第 i 个祖先为父节点的第 i-1 个祖先
    else
        parent[node][i] = -1;
}

for (int child : tree[node]) { // 遍历子节点
    depth[child] = depth[node] + 1; // 计算子节点的深度
    dfs(child, node); // 递归处理子节点
}
}

// 求解两个节点的最近公共祖先 (LCA)
// 时间复杂度为 O(log n)
int lca(int u, int v) {
    if (u == v) return u; // 如果两个节点相同，直接返回
    if (depth[u] < depth[v]) swap(u, v); // 确保 u 的深度小于 v 的深度
    int diff = depth[u] - depth[v]; // 计算深度差
    if (diff > 0)
        u = parent[u][diff - 1]; // 向上跳 v 层
    if (u == v) return u; // 如果 u 和 v 相同，说明祖先为 v 本身
    for (int i = MAX_LOG - 1; i >= 0; --i) { // 向上跳不同层
        if (parent[u][i] != parent[v][i]) {
            u = parent[u][i];
            v = parent[v][i];
        }
    }
    return parent[u][0];
}

```

2.5.5 调试分析（遇到的问题 and 解决方法）

在实现最近公共祖先的算法时，遇到的主要问题是如何高效地找到两个节点的公共祖先。通过深度优先搜索（DFS）填充父节点信息和深度数组，确保了后续 LCA 算法查询公共祖先的高效性。

2.5.6 总结和体会

这道题让我加深了深度搜索算法的能力，学习了 lca 算法。通过实现这一算法，我对树的深度和节点关系有了更深刻的理解。这次实验增强了我的算法设计能力和对数据结构的应用能力，特别是在处理复杂查询时的效率。

2.6 求树的后序

2.6.1 问题描述

给出二叉树的前序遍历和中序遍历，求树的后序遍历

2.6.2 基本要求

输入：包含若干行，每一行有两个字符串，中间用空格隔开

同行的两个字符串从左到右分别表示树的前序遍历和中序遍历，由单个字符组成，每个字符表示一个节点。

字符仅包括大小写英文字母和数字，最多 62 个

输入保证一颗二叉树内不存在相同的节点

每一行输入对应一行输出：若给出的前序遍历和中序遍历对应存在一棵二叉树，则输出其后序遍历，否则输出 Error

2.6.3 数据结构设计

```
// 定义树节点结构体
struct TreeNode {
    char value;           // 节点值
    TreeNode* lchild;     // 左子节点指针
    TreeNode* rchild;     // 右子节点指针

    // 构造函数
    TreeNode(char val)
        : value(val), lchild(nullptr), rchild(nullptr) {}
};
```

2.6.4 功能说明（函数、类）

```
//通过前序和中序创建二叉树
bool Error;

TreeNode* createTree(char* preorder, char* inorder, int preorderSize, int inorderSize) {
    if (preorderSize == 0 || inorderSize == 0) {
        return nullptr;
    }

    TreeNode* root = new TreeNode(preorder[0]); // 创建根节点
    // 寻找根节点在中序遍历中的位置
    int index = 0, i;
    for (i = 0; i < inorderSize; i++) {
        if (inorder[i] == root->value) {
            index = i;
            break;
        }
    }
}
```

```

if (i == inorderSize) { // 根节点不存在于中序遍历中，输入错误
    Error = true;
    return nullptr;
}

// 递归创建左右子树
root->lchild = createTree(preorder + 1, inorder, preorderSize - 1, index);
root->rchild = createTree(preorder + index + 1, inorder + index + 1, preorderSize - index - 1, inorderSize
- index - 1);

return root; // 返回根节点
}

```

2.6.5 调试分析（遇到的问题和解决方法）

在实现通过前序和中序遍历构建后序遍历的过程中，遇到的主要问题是如何正确地找到根节点在中序遍历中的位置。通过前序遍历来分割中序序列，再通过递归算法构建二叉树。

2.6.6 总结和体会

通过这道题，我对树的遍历和构建有了更深入的理解。前序和中序遍历的结合让我认识到树结构的动态性和复杂性。这次实验不仅提高了我的编程能力，也增强了我对树结构的理解和应用能力。

2.7 表达式树

2.7.1 问题描述

任何一个表达式，都可以用一棵表达式树来表示。例如，表达式 $a+b*c$ ，可以表示为如下的表达式树：

```

+
/ \
a  *
   / \
  b  c

```

现在，给你一个中缀表达式，这个中缀表达式用变量来表示（不含数字），请你将这个中缀表达式用表达式二叉树的形式输出出来。

2.7.2 基本要求

输入格式：

输入分为三个部分。

第一部分为一行，即中缀表达式(长度不大于 50)。

中缀表达式可能含有小写字母代表变量（a-z），也可能含有运算符（+、-、*、/、小括号），不含有数字，也不含有空格。

第二部分为一个整数 n ($n \leq 10$)，表示中缀表达式的变量数。

第三部分有 n 行，每行格式为 C x，C 为变量的字符，x 为该变量的值。

输出格式：

输出分为三个部分，第一个部分为该表达式的逆波兰式，即该表达式树的后根遍历结果。占一行。

第二部分为表达式树的显示，如样例输出所示。

如果该二叉树是一棵满二叉树，则最底部的叶子结点，分别占据横坐标的第 1、3、5、7……个位置（最左边的坐标是 1），

然后它们的父结点的横坐标，在两个子结点的中间。

如果不是满二叉树，则没有结点的地方，用空格填充（但请略去所有的行末空格）。

每一行父结点与子结点中隔开一行，用斜杠 (/) 与反斜杠 (\) 来表示树的关系。

/出现的横坐标位置为父结点的横坐标偏左一格，\出现的横坐标位置为父结点的横坐标偏右一格。

也就是说，如果树高为 m ，则输出就有 $2m-1$ 行。

第三部分为一个整数，表示将值代入变量之后，该中缀表达式的值。需要注意的一点是，除法代表整除运算，即舍弃小数点后的部分。

同时，测试数据保证不会出现除以 0 的现象。

2.7.3 数据结构设计

```
// 定义树节点结构体
struct TreeNode {
    char val; // 节点值
    TreeNode* left; // 左子节点指针
    TreeNode* right; // 右子节点指针
    int depth; // 节点深度
    int start; // 节点起始位置

    // 构造函数
    TreeNode(char x) : val(x), left(NULL), right(NULL), depth(0), start(0) {}
};
```

2.7.4 功能说明（函数、类）

```
// 定义一个函数，接收一个字符串表达式，将其转换为逆波兰表达式
string Inserve_Polish(string exp) {
    stack<char> st; // 初始化一个字符栈，用于存储操作符
    string res; // 初始化一个字符串，用于存储逆波兰表达式的结果
    for (int i = 0; i < exp.length(); i++) { // 遍历输入表达式的每一个字符
        if (exp[i] == '(') { // 如果当前字符是左括号
            st.push(exp[i]); // 将左括号压入栈
        } else if (exp[i] == ')') { // 如果当前字符是右括号
            while (st.top() != '(') { // 弹出栈顶元素直到遇到左括号
                res += st.top(); // 将栈顶操作符加入结果字符串
                st.pop(); // 弹出栈顶操作符
            }
            st.pop(); // 弹出左括号
        } else if (priority(exp[i]) > 0) { // 如果当前字符是操作符
            while (!st.empty() && priority(exp[i]) <= priority(st.top())) {
                res += st.top(); // 将栈顶操作符加入结果字符串
                st.pop();
            }
            st.push(exp[i]);
        }
    }
    res += st.top(); // 将栈顶操作符加入结果字符串
    st.pop();
    return res;
}
```

```

        st.pop();          // 弹出栈顶操作符
    }

    st.push(exp[i]); // 将当前操作符压入栈
} else {           // 如果当前字符是操作数
    res += exp[i];  // 直接将操作数加入结果字符串
}

}

}

while (!st.empty()) { // 将栈中剩余的操作符弹出
    res += st.top();  // 将栈顶操作符加入结果字符串
    st.pop();         // 弹出栈顶操作符
}

return res; // 返回转换后的逆波兰表达式
}

```

// 定义一个函数，接收一个字符串表达式，构建表达式树

```

TreeNode* BuildTree(string exp)
{
    stack<TreeNode*> operands; // 初始化一个栈，用于存储操作数（节点）
    stack<char> operators;     // 初始化一个栈，用于存储操作符

    for (int i = 0; i < exp.length(); i++) { // 遍历表达式中的每一个字符
        if (exp[i] == '(') { // 如果当前字符是左括号，直接将左括号压入操作符栈
            operators.push(exp[i]);
        } else if (exp[i] >= 'a' && exp[i] <= 'z') { // 如果当前字符是操作数
            operands.push(new TreeNode(exp[i])); // 创建一个新节点并压入操作数栈
        } else if (exp[i] == ')') { // 如果当前字符是右括号
            while (operators.top() != '(') { // 处理直到遇到左括号
                TreeNode* right = operands.top(); // 获取右操作数
                operands.pop(); // 弹出右操作数
                TreeNode* left = operands.top(); // 获取左操作数
                operands.pop(); // 弹出左操作数
                char op = operators.top(); // 获取操作符
                operators.pop(); // 弹出操作符
                TreeNode* node = new TreeNode(op); // 创建一个新的操作符节点
                node->left = left; // 设置左子树
                node->right = right; // 设置右子树
                operands.push(node); // 将新的子树压入操作数栈
            }
            operators.pop(); // 弹出左括号
        } else if (priority(exp[i]) > 0) { // 如果当前字符是操作符
            while (!operators.empty() && operators.top() != '(' && priority(exp[i]) <= priority(operators.top()))
            { // 优先级处理
                //和上面一样，弹出操作数，创建操作符节点，设置子树，压入操作数栈
            }
            operators.push(exp[i]); // 将当前操作符压入操作符栈
        }
    }
}

```

```

    } }

    while (!operators.empty()) {           // 处理剩余的操作符

        //和上面一样，弹出操作数，创建操作符节点，设置子树，压入操作数栈

    }

    return operands.top(); // 返回表达式树的根节点
}

// 计算给定的逆波兰表达式并返回结果
int Calculate(string inserve_polish, int val[]) {

    stack<int> operands;

    for (char c : inserve_polish) {

        if (c >= 'a' && c <= 'z') {

            operands.push(val[c - 'a']); //变量对应的值入栈

        } else {

            int right = operands.top(); //右操作数

            operands.pop();

            int left = operands.top(); //左操作数

            operands.pop();

            switch (c) { //运算符号

                case '+':

                    operands.push(left + right); //计算结果入栈，下同

                    break;

                case '-':

                case '*':

                case '/':

            } } }

            return operands.top();

        }
    }

```

2.7.5 调试分析（遇到的问题解决方法）

在实现逆波兰表达式的转换和计算时，主要问题在于操作符的优先级处理和括号的匹配。通过栈结构的辅助和运算符优先级的比较，可以正确求出逆波兰表达式，再通过逆波兰表达式简单的求出算式的值。

2.7.6 总结和体会

表达式树实际上是一种完全二叉树，每个叶子节点是运算数，中间节点都是操作符。在认识到这一点后就能顺利的构建出表达式树了：将每一个小的运算组合看成新的运算数，连接到上一层的运算符上，通过递归算法就可以构建。

3. 实验总结

本次实验一共完成了二叉树的非递归遍历、树的同构判断、感染二叉树的时间计算、树的重构、最近公共祖先的求解、树的后序遍历、求表达式树 7 道题目。这几道题目涉及的树的构建方

法和实现过程、应用等都各不相同，考验了我对二叉树知识的全面理解和掌握。

在本次实验中，我深入探索了多种树形数据结构及其相关操作，涵盖了二叉树、二叉搜索树、完全二叉树，有序树等多种类型，并熟悉了树初始化，节点连接，求树的深度，求树的前中后序遍历等多种关于二叉树的操作。

通过实现各种算法，比如深度搜索、广度搜索、LCA 算法、递归算法等等，我不仅加深了对树结构的理解，还提高了编程能力和算法设计能力。

同时，在实现二叉树的各种过程中，有时还需要其他数据结构来辅助构建实现，比如栈和队列、有序表等等，在这过程中，我将多种数据结构联系在一起，深入体会到各种数据结构之间的联系。我意识到，多种数据结构的有效组合可以极大的提高算法的效率。